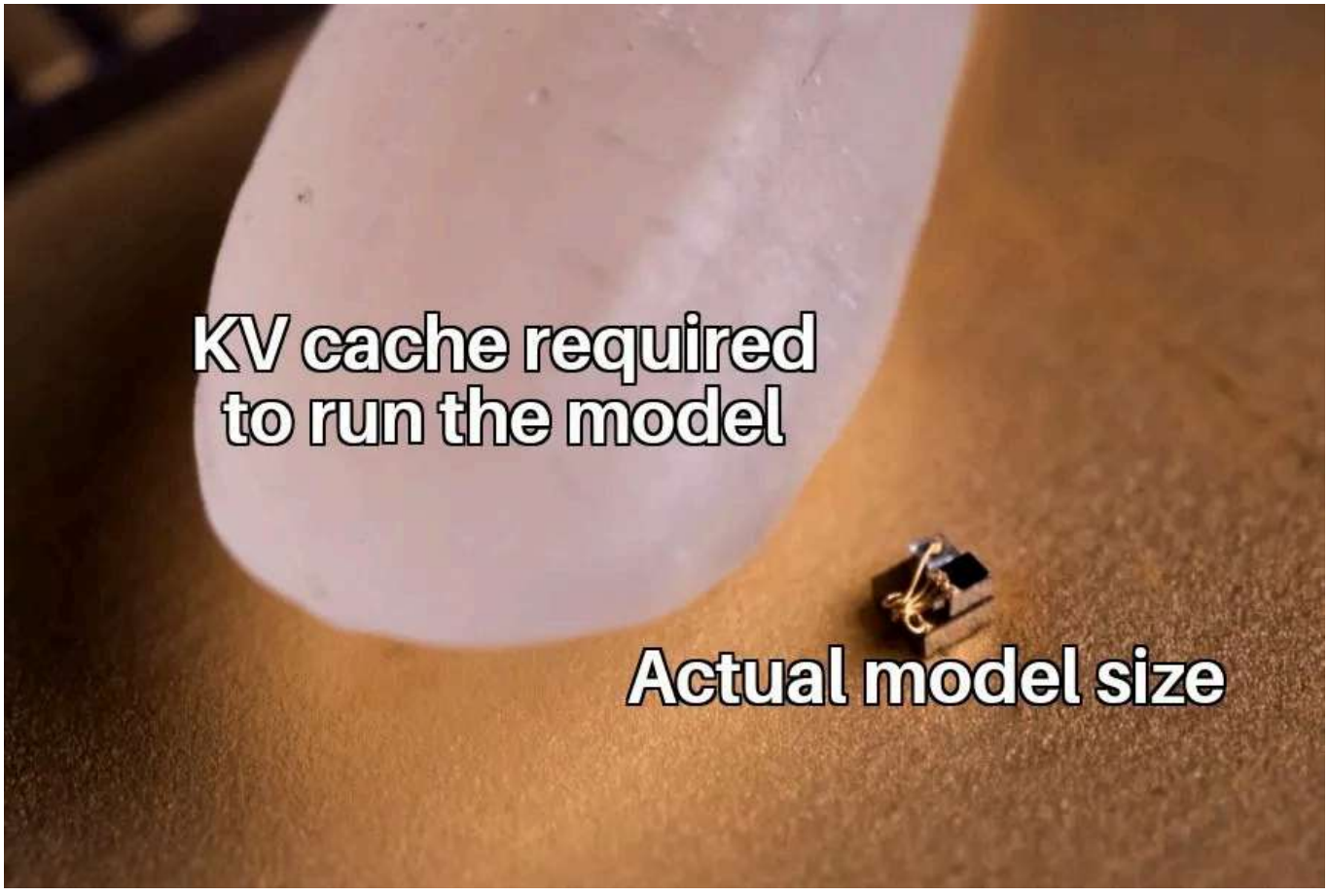
Breaking the Memory Wall: “Free” Model Depth with Group Tied Attention (GGTA)

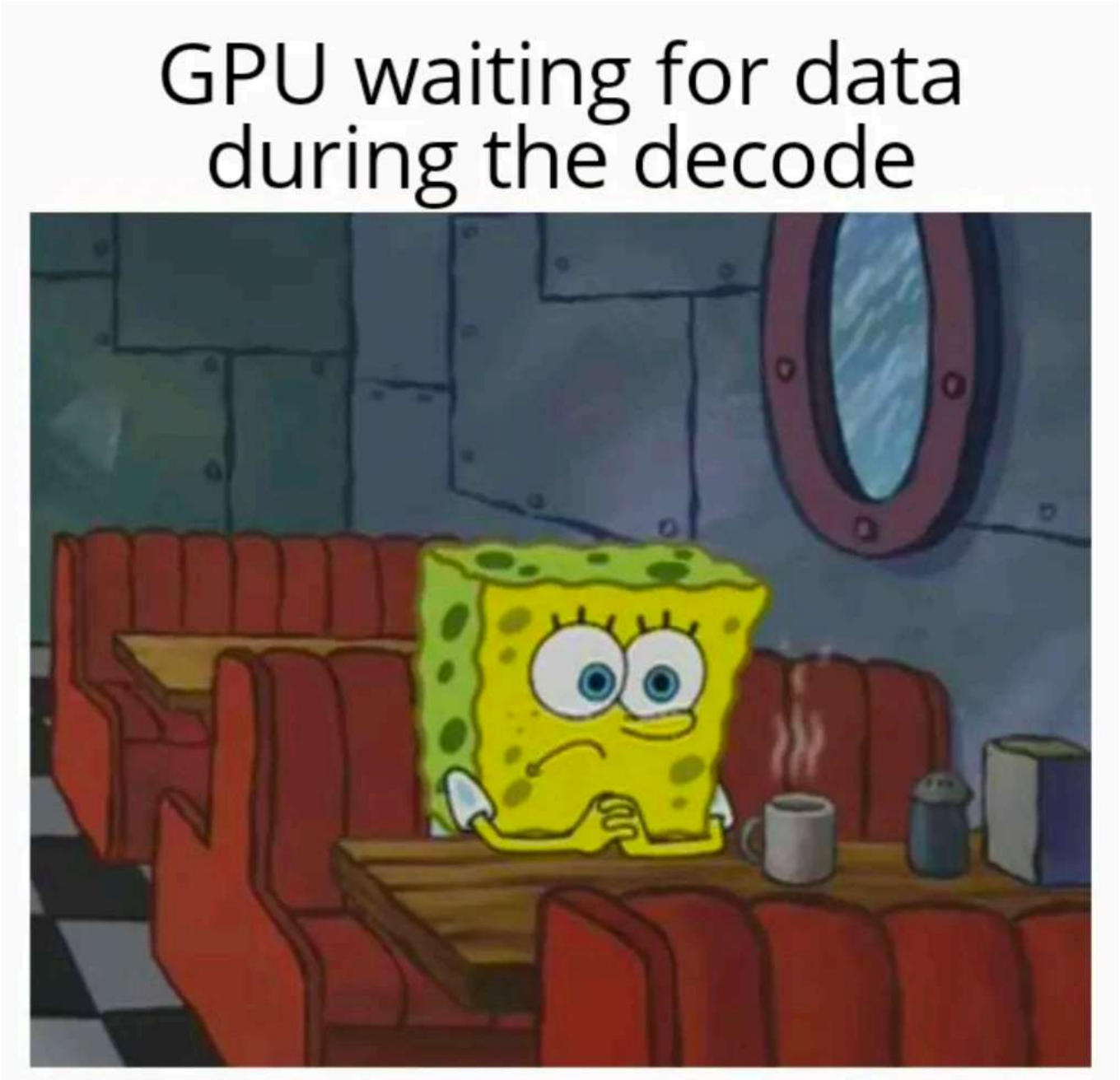
Chinmay Kulkarni 20 min read · 23 hours ago



Introduction: The Inference Bottleneck

When we talk about scaling Large Language Models (LLMs), the conversation usually revolves around massive GPU clusters, and trillion-token training runs. Training is undeniably compute-bound. But once a model is trained and deployed into production, the physics of the problem fundamentally change. Inference — specifically, the autoregressive decoding phase — is rarely constrained by how fast the GPU can crunch numbers. Instead, it hits a brutal physical limitation: it is overwhelmingly **memory-bandwidth bound**.

The primary offender is the standard Key-Value (KV) cache. During text generation, an LLM must “remember” the context of all previously processed and generated tokens. To avoid recalculating this context from scratch at every step, the model caches the Key and Value tensors for every attention layer. However, for every single new token generated, the hardware must physically haul this entire, continuously growing KV cache from High Bandwidth Memory (HBM) into the GPU’s SRAM. As the sequence length grows, the actual math operations (the matmuls) take a fraction of a millisecond, but waiting for the massive cache data to travel across the memory bus completely cripples the Time Per Output Token (TPOT).



This hardware bottleneck forces us into a difficult trade-off between sequence length, batch size, and total model depth. But what if we challenged the foundational architecture of the attention mechanism itself to break this wall?

What if we could completely eliminate the mathematical and physical distinction between “Keys” and “Values”?

By mathematically unifying these representations, we could drastically shrink the memory footprint at runtime. More importantly, the VRAM saved by compressing the cache doesn’t just sit idle — it can be directly reinvested into deploying deeper, smarter models without incurring the traditional latency penalties.

The Baseline Architectures

Before we can appreciate the mechanics of Group Tied Attention, we have to look at the baseline architectures we benchmarked it against. The evolution of attention mechanisms over the last few years has largely been a war against the KV cache.

Vanilla Multi-Head Attention (MHA)

Vanilla MHA is the original attention mechanism introduced in the seminal *Attention Is All You Need* paper. In this setup, the architecture is perfectly symmetric: if your model has 16 Query heads, it also has 16 Key heads and 16 Value heads.

While this provides maximum expressivity, the memory footprint is catastrophic during autoregressive decoding. For every single token in the sequence, the model must store a Key vector and a Value vector for every single head. This means the memory required for the KV cache scales linearly with the sequence length.

If you want to double your context window, you double your VRAM requirement. At shorter lengths (e.g., 512 tokens), this overhead is manageable. But as we push toward 4K, 8K, or 128K context windows, Vanilla

MHA causes the memory footprint to explode, leaving almost no room on the GPU for actual batching.

Grouped Query Attention (GQA)

To solve the MHA memory crisis, the industry pivoted to **Grouped Query Attention (GQA)**. Today, GQA is the dominant workhorse powering state-of-the-art models like Llama 3 and Mistral.

Instead of a 1-to-1 ratio, GQA forces multiple Query heads to share a single pair of Key and Value heads. For example, in our 24-layer baseline model, we use 16 Query heads but only 4 KV heads. This 4:1 ratio instantly provides a 4x reduction in the size of the KV cache compared to Vanilla MHA. The memory savings are massive, and the empirical degradation in model quality is minimal.

However, GQA still leaves a fundamental inefficiency on the table. While it successfully reduces the *number* of heads we need to store, it does not challenge the *nature* of the tensors themselves. GQA still fundamentally relies on storing two entirely distinct representations in memory: a Key tensor (used to compute the attention weights) and a Value tensor (used to compute the final output).

That means two separate memory allocations, two separate buffers to update at every step, and two massive tensors to drag across the memory bus.

This raises the million-dollar question: Do we actually need two?

The Innovation: Group Tied Attention (GTA & GGTA)

To solve the memory bandwidth crisis, we have to look at the problem through the lens of hardware utilization — specifically, a metric known as **Arithmetic Intensity**.

Arithmetic intensity measures the ratio of arithmetic operations to bytes of memory accessed (FLOPs per byte). During the sequential token-by-token decoding phase, Vanilla MHA achieves an arithmetic intensity of roughly 1:1. For every 2 bytes of the KV cache loaded (a `bfloat16` value), the GPU performs exactly 2 FLOPs (one multiply-accumulate). This is simplified to some extent, but I hope you understand what my intent is.

This means the GPU’s Tensor Cores sit effectively idle, starved for data, while waiting for the massive KV cache to cross the memory bus. GQA improves this by reusing a loaded KV head across a group of queries, increasing arithmetic intensity proportionally to the group size, but it still maintains the strict separation of Keys and Values.

The GTA Breakthrough: Unifying the Cache

The recent paper *“Hardware-Efficient Attention for Fast Decoding”* (Zadouri et al., 2025) introduced **Grouped-Tied Attention (GTA)** to attack this exact inefficiency.

Instead of two massive projections, GTA ties the Key and Value representations into a single shared state. In the paper’s implementation, the Value utilizes the full dimension of this tied state. For the Keys, only half of the vector comes from the unrotated tied state; the other half is generated by a separate, single-head projection where RoPE is applied, which is then broadcast and concatenated.

By loading a single state into on-chip memory and reusing it for both Keys and Values across a group of query heads, the authors proved that GTA roughly doubles the arithmetic intensity and halves the KV cache footprint relative to GQA, without compromising model quality.

Our Custom Implementation: Stripping it Down with ALiBi

While the original paper relies on a decoupled, partial RoPE projection, our JAX/Flax implementation pushes the GTA concept to be even leaner. We completely abandon dynamic positional states in the cache.

Here is how the math unfolds in our forward pass:

1. **The Projection:** The input is projected into Queries (Q) and a single, unified Answer tensor (A). Both use native `bfloat16`.
2. **The Attention Scores:** Instead of computing Q@K^T, we calculate the attention logits directly via the dot product of Q and A^T.
3. **Compile-Time ALiBi:** Rather than applying RoPE, we utilize ALiBi (Attention with Linear Biases). We compute the distance between the query and the cache positions, multiply it by compile-time constant slopes, and add the penalty directly to the attention logits. This allows XLA to constant-fold the positional math, requiring zero per-step runtime allocations.
4. **The Output:** Finally, the resulting attention weights are multiplied directly back against the *exact same* A tensor.

A acts as the Key during the logit calculation and as the Value during the output calculation. Maximum expressivity, half the data.

Pushing the Limits: Group Tied Attention Grouped (GGTA)

If tying K and V halves the memory footprint of Vanilla MHA, the absolute extreme of this design space is to apply GQA-style head reduction to our already unified tensor. I call this ultimate evolution **Group Tied Attention Grouped (GGTA)**. God knows why I came up with this weird name, but I hope you get my motivation behind naming it like that.

In GGTA, we aggressively compress the head dimensions:

1. **Extreme Head Reduction:** We project the Q tensor using the full complement of heads (e.g., `num_heads = 16`), but we strictly limit the A tensor projection to a dramatically reduced group count (e.g., `num_a_heads = 4`).
2. **The Hyper-Compressed Cache:** This generates an A cache of shape `[batch, num_a_heads, max_len, depth]`. Because it is tied *and* grouped, this cache is 4x smaller than our standard GTA implementation, and

significantly smaller than GQA's paired (K, V) cache at the identical head count.

3. **Zero-Copy Expansion:** To compute the attention scores, the math requires the head counts to match. During the forward pass, we expand the compressed A tensor from 4 heads to 16 heads using a `jnp.broadcast_to` operation. This is a zero-copy view, meaning the JAX/XLA compiler expands the tensor dynamically during computation without allocating any new memory.

```
# Reshape to introduce a broadcastable dimension for the query groups
a_use_exp = a_use.reshape(batch_size, self.num_a_heads, 1, max_len, depth)

# Zero-copy expansion to match the 16 Query heads
a_use_rep = jnp.broadcast_to(
    a_use_exp,
    (batch_size, self.num_a_heads, num_queries_per_a, max_len, depth)
).reshape(batch_size, self.num_heads, max_len, depth)
```

By using `jnp.broadcast_to` instead of an operation like `jnp.repeat`, we merely edit the strides and metadata of the tensor rather than allocating new physical memory. This creates a zero-copy view, meaning the JAX/XLA compiler fakes the expanded dimensions dynamically during the matrix multiplication without increasing our VRAM footprint.

GGTA is designed to maximize arithmetic intensity to its theoretical limit, reusing a heavily compressed, unified representation across multiple Query heads to wring every possible drop of utility out of the GPU’s memory bandwidth.

Depiction of the attention variants used in this project.

Why ALiBi and not RoPE in GTA and GGTA?

When unifying the Key and Value into a single tensor, handling positional encoding becomes a massive architectural headache.

If we wanted to use standard Rotary Positional Embeddings (RoPE) out of the box, we would run into a fundamental mathematical wall: **RoPE mutates the geometric space of the tensor to encode position, which ruins it for use as a Value vector.** Therefore, a naive RoPE implementation would force us to maintain two entirely separate tensors in the cache anyway — one “RoPE’d” tensor to act as the Key, and one “un-RoPE’d” tensor to act as the Value. This

completely defeats the entire purpose of a unified, single-tensor attention cache.

To get around this, the original GTA paper (Zadouri et al., 2025) had to implement a complex “partial” RoPE strategy. Their approach required taking a separate, single-head projection, applying RoPE to it, broadcasting it across all groups, and then concatenating it with the unrotated half of the tied tensor to form the final Key.

While this works on paper, from a systems engineering perspective in JAX/XLA, it is messy. Slicing, rotating, broadcasting, and concatenating tensors at every single decode step forces the compiler to allocate intermediate buffers, introducing memory overhead and computational friction that we are actively trying to eliminate.

To strip the architecture down to its absolute leanest form and protect our single-tensor design, I completely abandoned RoPE in favor of **ALiBi (Attention with Linear Biases)**.

ALiBi takes a fundamentally different approach: it does not touch the embeddings or the cache tensors at all. Instead of mathematically rotating the Query and Key vectors, ALiBi simply applies a static linear penalty directly to the attention logits before the softmax operation. The penalty is proportional to the distance between the Query token and the cached Key token, multiplied by a head-specific slope.

Choosing ALiBi for GTA and GGTA unlocked two massive engineering advantages:

1. **Pure Tensor Tying:** Because we no longer need to rotate any vectors, the unified A tensor remains 100% clean and untouched. We can project it once, cache it, and use it identically as both the Key (for the dot product) and the Value (for the output) without any complex slicing, duplication, or concatenation.
2. **Zero-Cost XLA Execution:** In our JAX implementation, the `num_heads` parameter is a compile-time attribute. Because ALiBi relies on static slopes and relative distances, hopefully the XLA compiler constant-folds this entire operation across all decode steps. There are absolutely no dynamic positional states materialized or allocated at runtime, keeping our memory mutations strictly O(1).

By swapping RoPE for ALiBi, we bypassed the need to maintain duplicate RoPE’d states, removing the final piece of overhead standing in the way of a truly hyper-compressed, maximum-throughput attention mechanism.

Systems Engineering: Building a Rigorous JAX Benchmark

Theoretical memory savings mean nothing if the underlying compiler and hardware cannot execute the architecture efficiently. To prove the viability of GGTA, we built a highly isolated profiling suite entirely from scratch using JAX and Flax.

Measuring the true runtime performance of an attention mechanism requires stripping away framework noise and carefully managing how the XLA (Accelerated Linear Algebra) compiler handles memory. Here is how we engineered the benchmark to guarantee pristine hardware measurements.

The Noise Problem and Strict Incremental Tracking

When benchmarking Large Language Models, standard memory profilers are notoriously noisy. They capture the entire memory state of the GPU, which includes the CUDA context, the JAX runtime buffers, and the massive allocations required for the static model weights themselves. This background noise easily obscures the precise footprint of the attention mechanism we are trying to measure.

To solve this, our benchmark implements strict incremental VRAM tracking. After fully initializing the model and compiling the graphs, but before starting the actual profiling loop, we capture a `post_warmup_bytes` baseline directly from the local device's memory statistics. During the actual inference run, we subtract this baseline from the peak memory usage. This isolates the incremental cost of the profiled computation, ensuring the reported VRAM reflects *only* the dynamically allocated KV or A caches and the active memory mutations.

Graph Warmups and Preventing TTFT Inflation

JAX operates on a Just-In-Time (JIT) compilation model, where Python code is traced and compiled into optimized GPU kernels on the fly. This compilation process carries a heavy tax — absorbing roughly some time for the prefill graph and yet some more time for the decode graph in our tests.

If a performance timer starts during this compilation phase, the Time To First Token (TTFT) metrics will be massively and artificially inflated. To prevent this, we explicitly warm up both the `prefill_step` and `decode_step` graphs before starting the timers.

Furthermore, JAX dispatches GPU work asynchronously. A Python call can return immediately while the GPU is still executing cache writes in the background. Measuring true Time To First Token (TTFT) requires isolating

the attention mechanism’s performance from the constant overhead of the rest of the model. Because JAX dispatches work to the GPU asynchronously, we have to use `jax.block_until_ready` strategically to stop our timers at the exact right millisecond.

During the **Prefill step** (measuring TTFT), we explicitly block *only* on the KV caches:

```
# Actual prefill
logits, active_caches = prefill_step(variables, prompt_input, active_caches)

# Block ONLY on the KV cache, ignoring the final argmax and logit projection
jax.block_until_ready(active_caches)
```

Why do we do this? Multiplying the final output vector against a massive 50K-word vocabulary matrix takes the exact same amount of time regardless of whether we use MHA, GQA, or GGTA. By stopping the timer the moment the cache is filled — deliberately leaving the vocabulary projection out of the TTFT measurement — we strip away that constant overhead. This allows the raw performance delta between our specific attention variants to stand out clearly in the data.

However, during the **Decode step** (measuring TPOT), the math changes. Because autoregressive generation is a strict sequential loop, Step 2 mathematically cannot exist until Step 1 provides the exact next token. Therefore, we must block on both the cache update *and* the logits to capture the full cycle time:

```
# Actual decode
logits, active_caches = decode_step(variables, next_token, pos, active_caches)

# Block on BOTH to capture the full sequential cycle
jax.block_until_ready((logits, active_caches))
```

Precision Policies and O(1) Memory Mutations

To maximize decoding throughput without sacrificing model accuracy, we implemented two crucial low-level optimizations:

LayerNorm-Aware Precision: Running inference in `bfloat16` is standard practice for speed, but applying it blindly can cause catastrophic numerical degradation. Variance accumulation inside LayerNorm is highly sensitive to precision loss. We wrote a custom `tree_map_with_path` casting policy that converts all dense projections (like embeddings and matmuls) natively to `bfloat16`, while strictly shielding LayerNorm scales and biases, keeping them in `float32`. This ensures maximum throughput without corrupting the normalization quality.

O(1) In-Place Memory Mutations: Because JAX arrays are mathematically immutable by default, naively appending tokens to the cache (like using `jnp.concatenate`) forces the compiler to allocate entirely new, larger tensors at every single step. Dragging a continuously growing tensor across the GPU memory bus instantly destroys your bandwidth.

To solve this, we pre-allocate the maximum necessary cache size upfront and separate this state from our logic using Flax’s `variables` dictionary. Then, we update it using XLA's lowest-level mutation operation:

```
# active_caches is our pre-allocated buffer: [batch, num_heads, max_len, depth]
# tied_a is the newly generated token's representation: [batch, num_heads, 1, de

# Force an O(1) in-place memory overwrite at the current sequence 'pos'
active_caches = jax.lax.dynamic_update_slice(
    active_caches,
    tied_a,
    (0, 0, pos, 0) # Write starting at indices: (batch 0, head 0, current pos, d
)
```

By utilizing `jax.lax.dynamic_update_slice` inside our decode step, we guarantee that the XLA compiler performs a true O(1) in-place memory mutation. It writes the new token's data directly into the exact correct slot of the pre-allocated VRAM buffer. This ensures that the massive attention matrices never re-materialize or copy themselves inefficiently during the autoregressive decode phase.

The Experiment Matrix

To rigorously evaluate our implementations, we designed an automated benchmarking environment controlled by a master shell script (`run_benchmarks.sh`). Rather than running tests in a linear, predictable sequence—which can introduce systematic errors due to hardware caching or progressive thermal degradation—our benchmarking harness is engineered to meet scientific standards.

The Benchmarking Harness and Thermal Control

The master script executes a matrix of 14 distinct experiments across four classic sequence lengths: 512, 1024, 2048, and 4096 tokens. For each

evaluation run, the script bundles all possible (Experiment, Sequence Length) tuples, shuffles them completely using the `shuf` utility, and executes them in a fully randomized order. This eliminates any temporal bias from the profiling data.

Crucially, after every single isolated run, the script forces the system to pause. It triggers a strict **3-minute thermal sleep** to let the GPU drop back down to its baseline idle temperature before initiating the next configuration. This deliberate cool-down period prevents the hardware from hitting thermal limits and dynamically throttling its clock speeds mid-test, ensuring every architecture is evaluated under identical, pristine hardware conditions.

Isolating the Variables: Two Analytical Frameworks

To make a truly fair architectural comparison, you cannot just look at model names; you have to isolate the specific hardware axes they alter. We structured our results around two distinct experimental frameworks.

1. The Iso-KV-Cache Narrative

Multi-Head Attention stores full head dimensions per layer, while Grouped Query Attention scales that down by your grouping ratio. In our baseline architecture with 16 Query heads and 4 KV heads, the ratio is exactly 4:1.

To isolate the cost of memory bandwidth from total computation, we designed **An Iso-KV-Cache MHA Baseline**. By scaling an MHA model down to just 6 layers, its total active KV cache size perfectly matches that of a 24-layer GQA model. This allows us to hold the memory axis completely constant. If the 24-layer GQA model performs differently, we know the delta is driven by the computation axis (layer depth), not memory saturation.

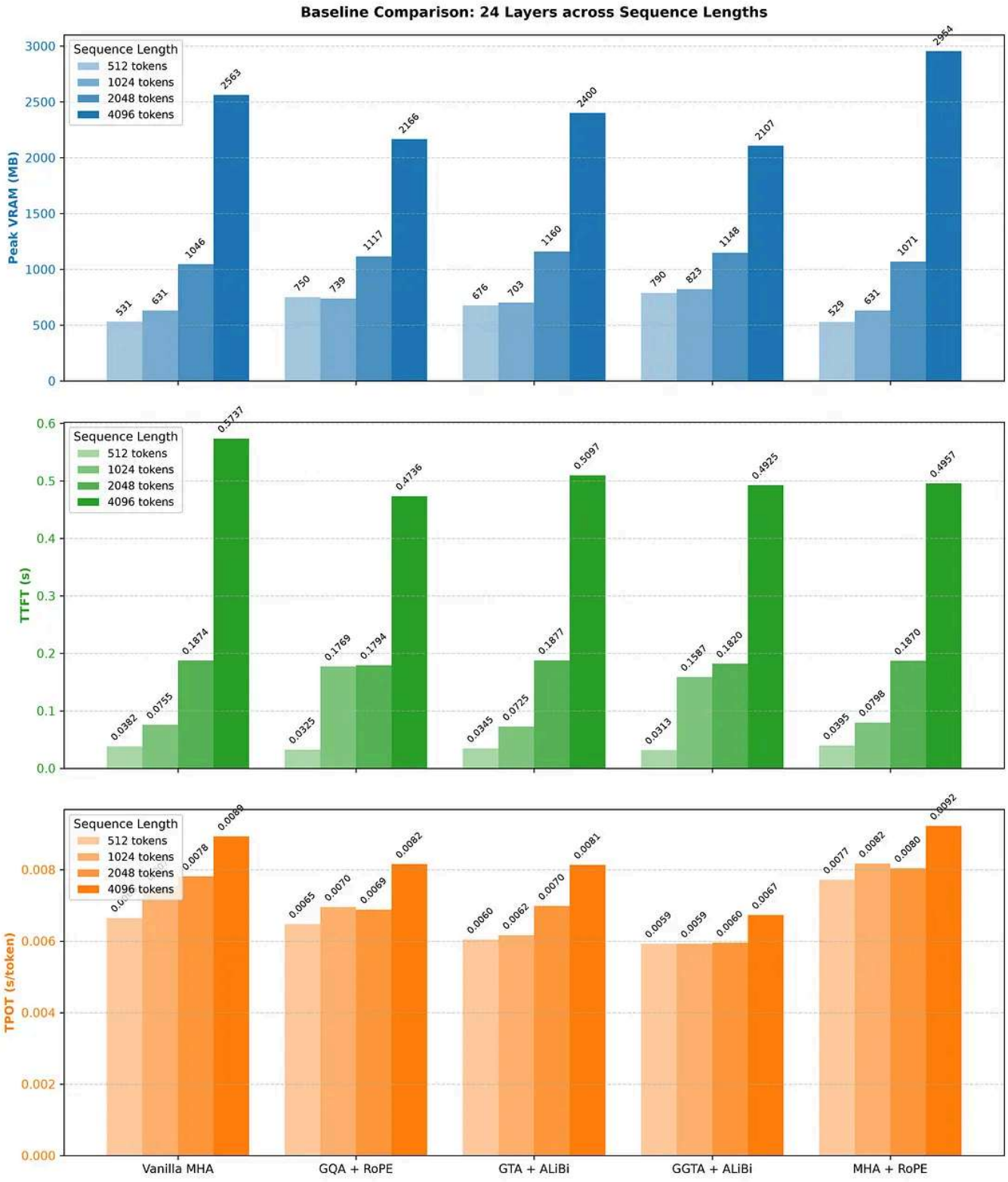
2. The Iso-Parameter Narrative (Holding the Parameter Budget Constant)

When you strip out heads or tie representations together, you naturally lose learned parameters and representational capacity from the attention layers. To ensure our comparisons hold water, we use an **Iso-Parameter Framework**. Our anchor is **Vanilla MHA at 24 Layers (~354.6M parameters)**. As we move to more compressed variants like GGTA, we can choose to either match that parameter count exactly by expanding the FFN width at 24 layers, or — more excitingly — **reinvest the VRAM savings to add entire layers of compute** while staying within the same parameter and hardware memory budget. This dual-narrative setup lays the groundwork for the benchmark results. We do the latter.

Results:

Asymptotic VRAM Scaling

When evaluating attention mechanisms for production, the most critical metric for scalability is how the memory footprint reacts as the context window grows. In our benchmark, the peak incremental VRAM usage tells a clear story of asymptotic efficiency.



24-layer baseline data

The Data: Flattening the Curve

If we look at the 24-layer baseline data, Vanilla MHA behaves exactly as the underlying math dictates: linearly and aggressively. At a short 512-token

sequence, MHA consumes roughly 531 MB of VRAM. But as we push the context to 4096 tokens, that footprint explodes to over 2563 MB.

By contrast, GGTA successfully breaks this linear trajectory. At 4096 tokens, the 24-layer GGTA model requires only ~2107 MB of VRAM. By simply unifying the Key/Value representations and grouping the heads, we shaved off over 450 MB of VRAM on a relatively small model at a modest 4K context. As a calculated guess, as sequence lengths scale toward 32K, 128K, or beyond, this gap will only widen, saving gigabytes of precious HBM.

The Reality Check (The Low-Context Tax)

However, systems engineering requires complete transparency, and there is a fascinating nuance in the data at low context lengths.

If you look closely at the 512-token mark, you will notice a “low-context tax.” At 512 tokens, Vanilla MHA is actually the most memory-efficient architecture (~531 MB), while GGTA consumes significantly more (~790 MB). Why does the “efficient” architecture use more memory for short prompts? This inversion is a classic symptom of framework and compiler mechanics. The zero-copy `broadcast_to` operation we use to expand the heavily compressed A tensor establishes a higher fixed baseline memory floor inside the XLA compiler. There are also static buffer allocations and padding heuristics that penalize complex tensor routing at microscopic scales. MHA avoids this overhead simply because its memory layout is perfectly contiguous and straightforward. There is also ALiBi that can cause this VRAM to go up in usage. However this is what I understood from the code. If you think there is a different reason for this behaviour, do let me know.

But for modern LLM deployments, a 512-token context is practically obsolete. The battleground for AI hardware is in long-context retrieval, RAG

(Retrieval-Augmented Generation), and complex agentic reasoning. In these domains, the low-context tax is completely eclipsed by GGTA’s ability to aggressively flatten the VRAM scaling curve.

While VRAM savings dictate whether a model can physically fit on a GPU, the ultimate measure of inference efficiency — and the metric that directly impacts user experience — is the Time Per Output Token (TPOT) during the autoregressive decode phase. Because decoding is heavily memory-bandwidth bound, smaller caches should theoretically translate to faster generation.

Looking at the benchmark data, GGTA does not just validate this theory; it absolutely dominates the field.

The Data: Breaking the Speed Limit

As the sequence length grows and the memory bus becomes increasingly saturated, the decode speeds naturally degrade. At the maximum evaluated sequence length of 4096 tokens, our 24-layer Vanilla MHA baseline struggles under the weight of its massive cache, clocking in at a median TPOT of 0.0089 seconds per token.

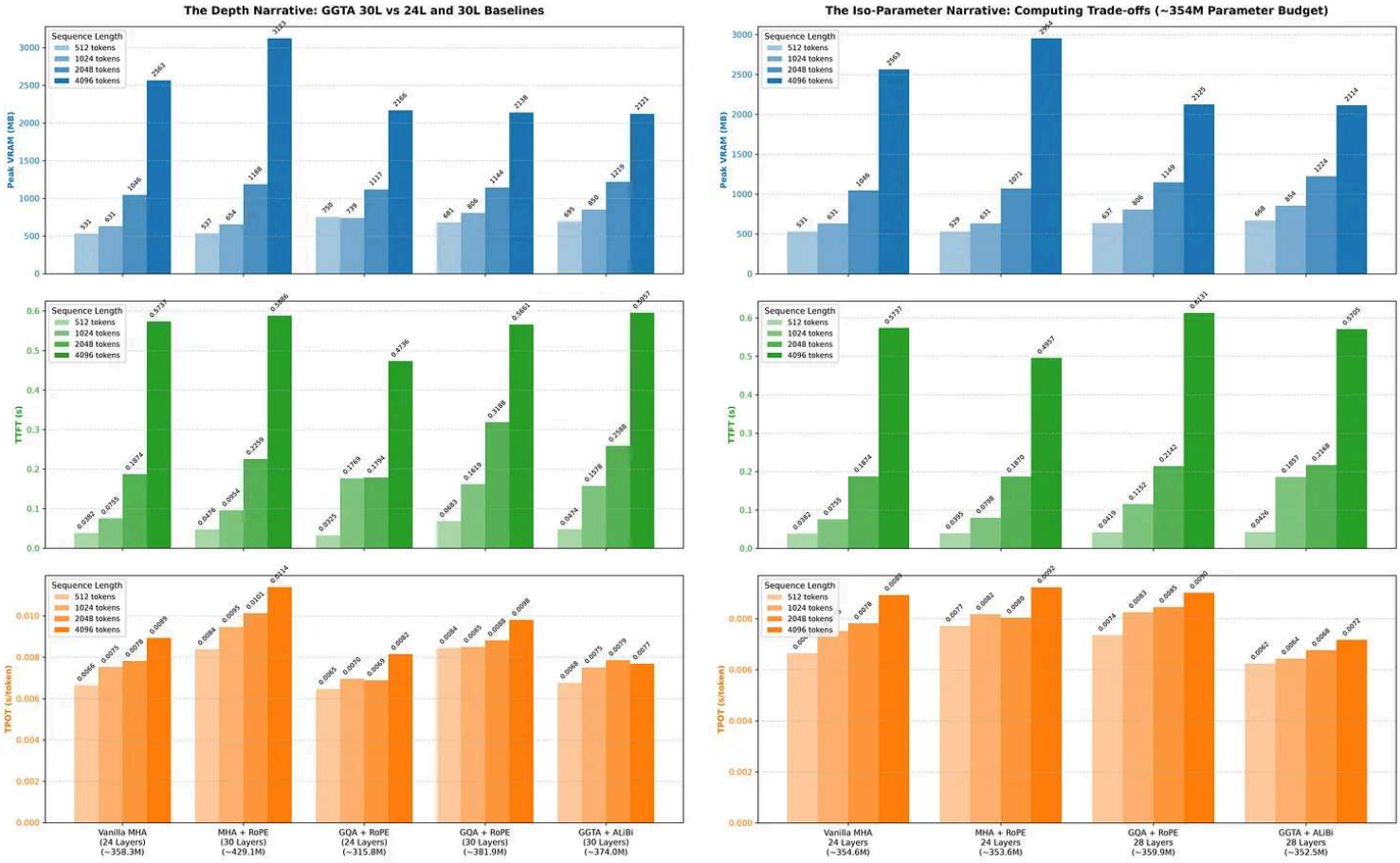
Grouped Query Attention (GQA) at 24 layers successfully relieves some of this memory pressure, bringing the TPOT down to 0.0082 seconds per token.

However, the crowning achievement of this benchmark is the performance of the 24-layer GGTA model. Under the exact same context conditions at 4096 tokens, GGTA achieves a blistering TPOT of just **0.0067 seconds per token**.

The Comparison

To put that into perspective, GGTA delivers an 18% speedup over GQA and a nearly 25% speedup over Vanilla MHA.

This speed advantage is the direct result of maximizing arithmetic intensity. By unifying the Key and Value into a single tied A tensor and aggressively grouping it, GGTA forces the hardware to fetch significantly less data per generated token. Instead of waiting on the memory bus, the GPU’s Tensor Cores stay continuously fed, resulting in a dramatically faster, more responsive model without sacrificing the 24-layer architectural depth.



The first image depicts a deeper model with GGTA compared to baseline. The second image depicts performance of GGTA with fixed parameter budget.

The “Free” Depth Advantage (Iso-Parameter)

In modern LLM architecture, there is a constant tug-of-war between parameter count, VRAM limits, and inference speed. When you heavily compress the attention mechanism — as we do in GGTA — you inevitably strip away learned parameters from the model.

To ensure our performance gains aren’t just a byproduct of running a “smaller” model, we have to look at the Iso-Parameter comparison. If we have a strict parameter budget, what is the most efficient way to spend it?

The Trade-off: Trading Width for Depth

For this analysis, our budget is roughly ~354 million parameters. Our anchor is the Vanilla MHA model at 24 layers, which comes in at ~354.6M parameters.

Because GGTA’s unified and grouped cache is so lightweight, we can take the parameters saved from the attention layers and reinvest them into the model’s depth. By doing so, we built a 28-layer GGTA model that perfectly matches the budget at ~352.5M parameters.

We are effectively asking the hardware: *If we force you to do 4 additional layers of dense mathematical computation (FLOPs) per token, will the memory savings from the GGTA cache make up for the extra work?*

The Win: Deeper, Faster, Lighter

The answer is an unequivocal yes. The benchmark results prove that memory bandwidth is such a severe bottleneck that you can actually add “free” layers of compute by fixing the cache.

At a 4096-token sequence length:

- The **24-layer Vanilla MHA** model requires 2563 MB of VRAM and decodes at **0.0089 seconds** per token.
- The **28-layer GGTA** model requires only 2114 MB of VRAM and decodes at **0.0072 seconds** per token.

Let that sink in. Even with 4 additional layers of neural network to compute during every single forward pass, the GGTA model is still nearly 20% faster and consumes 450 MB less memory than the shallower baseline.

This is the holy grail of inference optimization. We are successfully trading memory-bound KV cache footprint for compute-bound model depth. The result is a model that is theoretically smarter (deeper) and fundamentally faster, all while sitting comfortably in a smaller hardware footprint.

Conclusion & Next Steps

The LLM memory wall is not an insurmountable hardware limitation; it is an architectural bottleneck. As our custom JAX/XLA benchmarks demonstrate, Group Tied Attention Grouped (GGTA) proves that we can break this wall. We no longer have to settle for merely sharing Key and Value heads — we can fundamentally tie their mathematical representations together. By collapsing the cache footprint, GGTA drastically increases the arithmetic intensity of autoregressive decoding, keeping the GPU’s Tensor Cores fully saturated and shattering the baseline Time Per Output Token (TPOT).

The Next Frontier: Training and Perplexity

Hardware efficiency, however, is only half the battle. A model that generates tokens at lightning speed means very little if its predictive accuracy

degrades.

The immediate next phase of this project is to train all of these architectural variants from scratch. By rigorously tracking the validation perplexity per training step, we will evaluate the pure predictive fidelity of both GTA and GGTA. The ultimate goal is to definitively prove whether these unified attention mechanisms are viable, zero-compromise replacements for the architectures powering today’s frontier models.

The Future of Inference

If the perplexity and downstream accuracy hold up, the implications for the broader AI ecosystem are profound.

For edge deployment and on-device AI, where VRAM is severely constrained, GGTA provides a pathway to fit far more capable models directly onto consumer hardware. For enterprise applications utilizing long-context LLMs, it means processing massive document retrievals without grinding the memory bus to a halt.

Ultimately, the ability to trade memory-bound cache footprints for “free” architectural depth proves that the future of inference isn’t just about building bigger GPU clusters. It is about engineering smarter, deeper, and fundamentally faster models that push the absolute limits of the silicon they run on.

Some applications that I can think of for these kinds of models are:

- 🗣️ **Real-Time Voice Assistants:** Think about conversational AI on smart speakers. The context is usually just a few back-and-forth sentences, but the Time To First Token (TTFT) needs to be nearly instant to feel natural.

- 💻 **Inline Code Autocomplete:** Tools like GitHub Copilot suggesting the next few lines of code while you type. They usually only need to read the active script (under a few thousand tokens), but they must generate text fast enough to keep up with a programmer’s typing speed.
- 🎮 **Massive Multi-Agent Simulations (Video Game NPCs):** Imagine a game running 1,000 AI characters simultaneously. Each character only needs a small memory buffer of recent events, but running 1,000 standard Multi-Head Attention KV caches concurrently would instantly exhaust a GPU’s VRAM.
- 🚦 **High-Volume API Routing:** Systems that instantly read an incoming customer support ticket and route it to the correct department. The text is short, but the server needs to process tens of thousands of these per second.

If you would like to explore the codebase, it is here: https://github.com/CAMKULKARNI/attention-exploration. I have tried to optimize the attention variants as much as I could for a fair comparison. However if you think that any variant is not in its optimal form, you can let me know in the comments.

So, like Chandler Bing does Statistical Analysis and Data re-configuration, I work on Machine learning and Deep learning 😅. If you liked this work, and would like to check out my other repositories, you can check them here: GITHUB. You can connect with me here: LinkedIn

See you in the next one...

PS: I am not claiming that this architecture or my literature survey is definitive. I am continuously learning and refining these designs. If you identify any shortcomings or have suggestions for the next iteration, please share them in the comments; I would love to discuss and learn from your perspectives.
Thank you so much for reading...

Unlisted

Written by Chinmay Kulkarni

3 followers · 22 following

Edit profile